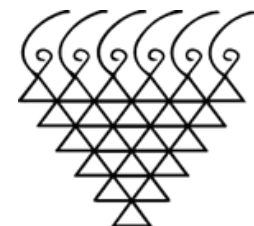


LLMOps

Operationalizing Large Language Models



TMLC
ACADEMY

What is LLMOps?

 Definition:

LLMOps (Large Language Model Operations) is the practice of managing, deploying, and maintaining LLMs efficiently in production.

 Why is it important?

- Bridges the gap between research and real-world applications
- Ensures scalability, reliability, and efficiency
- Addresses challenges like high compute costs and model drift

Evolution from MLOps to LLMOps

🔧 MLOps vs. LLMOps

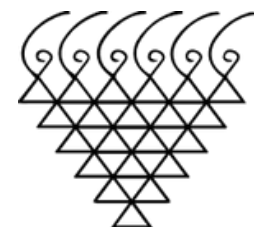
Feature	MLOps	LLMOps
Model Type	Traditional ML models	Large Language Models
Compute	Moderate	High (GPUs, TPUs)
Data Handling	Structured/tabular	Unstructured (text, code)
Monitoring	Evaluation Metrics & Drift	Token Usage, Latency, Hallucinations

💡 LLMOps extends MLOps with additional complexities for LLMs

Challenges in Deploying LLMs

Key Challenges:

- Compute & Cost  → Expensive GPUs & cloud resources
- Latency & Scalability  → Slow inference, need for optimization
- Security & Privacy  → Sensitive data handling, compliance (GDPR, HIPAA)
- Model Drift  → Performance degradation over time
- Hallucinations & Bias  → Unreliable responses



Core Components of LLMOps

LLMOps consists of five core components:

- Model (Pre-trained / Fine-tuned)
- Model Serving (LLM, RAG, Agentic)
- Search (Vector Search/Databases)
- Data I/O Management
- Infrastructure & Deployment Optimizations
- Monitoring & Observability

Model

🔧 Developing & Managing LLMs

Pre-trained vs. Fine-tuned Models

Open-source: LLaMA, Mistral, Falcon

Closed-source: GPT-4, Claude, Gemini

Fine-tuning Methods

LoRA, QLoRA, PEFT for cost-effective tuning

Choosing the Right Model

Trade-offs between accuracy, cost, and deployment feasibility



Model Serving

Optimizing LLM Inference

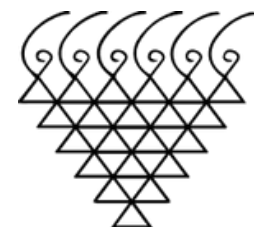
Inference Optimization Techniques: vLLM, DeepSpeed, TGI (Faster & cost-efficient inference)

Batch Inference & Token Efficiency - Reducing redundant computations

Deployment Choices - Cloud (AWS, GCP, Azure -> Server vs Serverless) vs. On-prem (Private GPUs, TPUs)

Streaming vs. Real-time Inference

When to use each approach for low-latency responses



TMLC
ACADEMY

Search (Vector Search & Databases)

Why is Search Important in LLMOps?

- Enables Retrieval of data
- Improves accuracy by grounding LLMs with external knowledge
- Reduces hallucinations and irrelevant responses

Vector Search Solutions

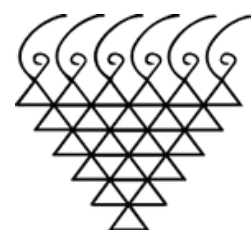
Pinecone, ChromaDB, FAISS, Weaviate, Qdrant, LanceDB

 Implementing caching of previous queries and responses for faster response times

Data I/O Management

Handling Data for LLMs

- Data Preprocessing & Cleaning
 - Ensuring high-quality input data
- Embedding Generation & Storage
 - Managing vectorized representations efficiently
- Structured vs. Unstructured Data
 - Optimizing for diverse data sources (text, logs, knowledge bases)
- Continuous Learning & Feedback Loops
 - Iteratively improving model performance
- Guardrails
 - To make sure the input data and response are ethical and bound to known factors

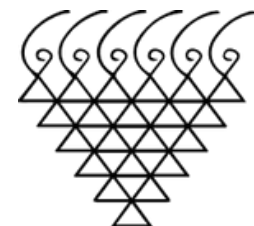


Infrastructure & Deployment



Scaling LLM Workloads Efficiently

- Hardware Selection:
 - GPUs, TPUs, custom accelerators
- Docker and Kubernetes
 - Efficient containerization
- AutoScaling
 - Horizontal and Vertical Scaling strategies on Cloud
- Cost Optimization Techniques:
 - Quantization (4-bit, 8-bit models)
- Load Balancing & Traffic Management
 - Managing requests efficiently under heavy loads
- CI/CD/CT
- WorkFlow Orchestration



TMLC
ACADEMY

Monitoring & Observability

What to Monitor?

- ✓ Token usage & latency
- ✓ Response accuracy & model drift
- ✓ Security threats & data privacy risks

Key Monitoring Tools

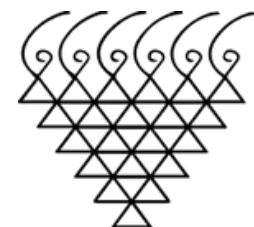
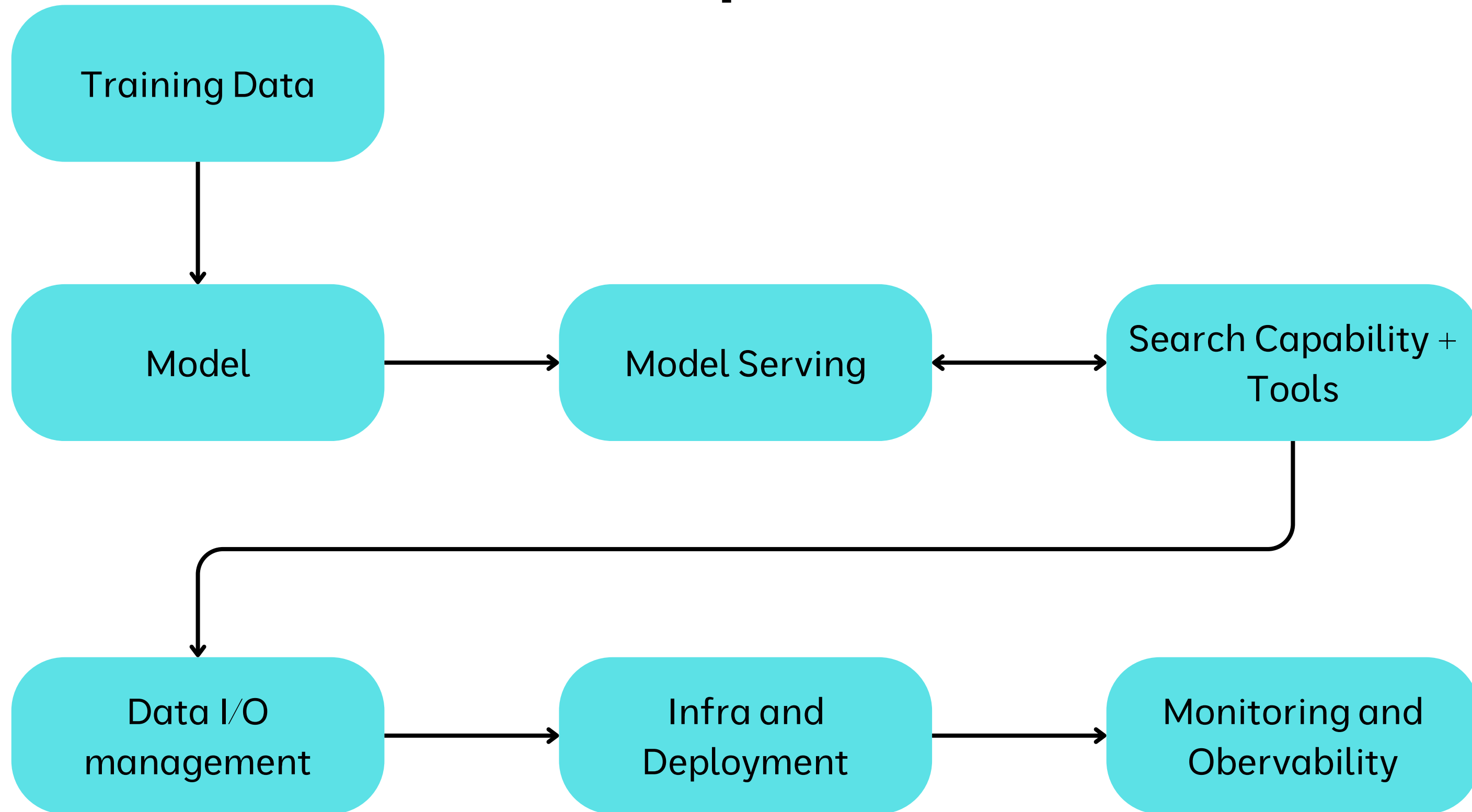
Langsmith, Opik, LangFuse

Prometheus & Grafana

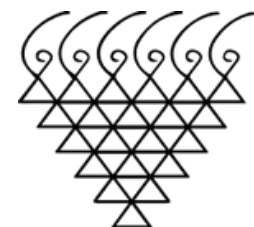
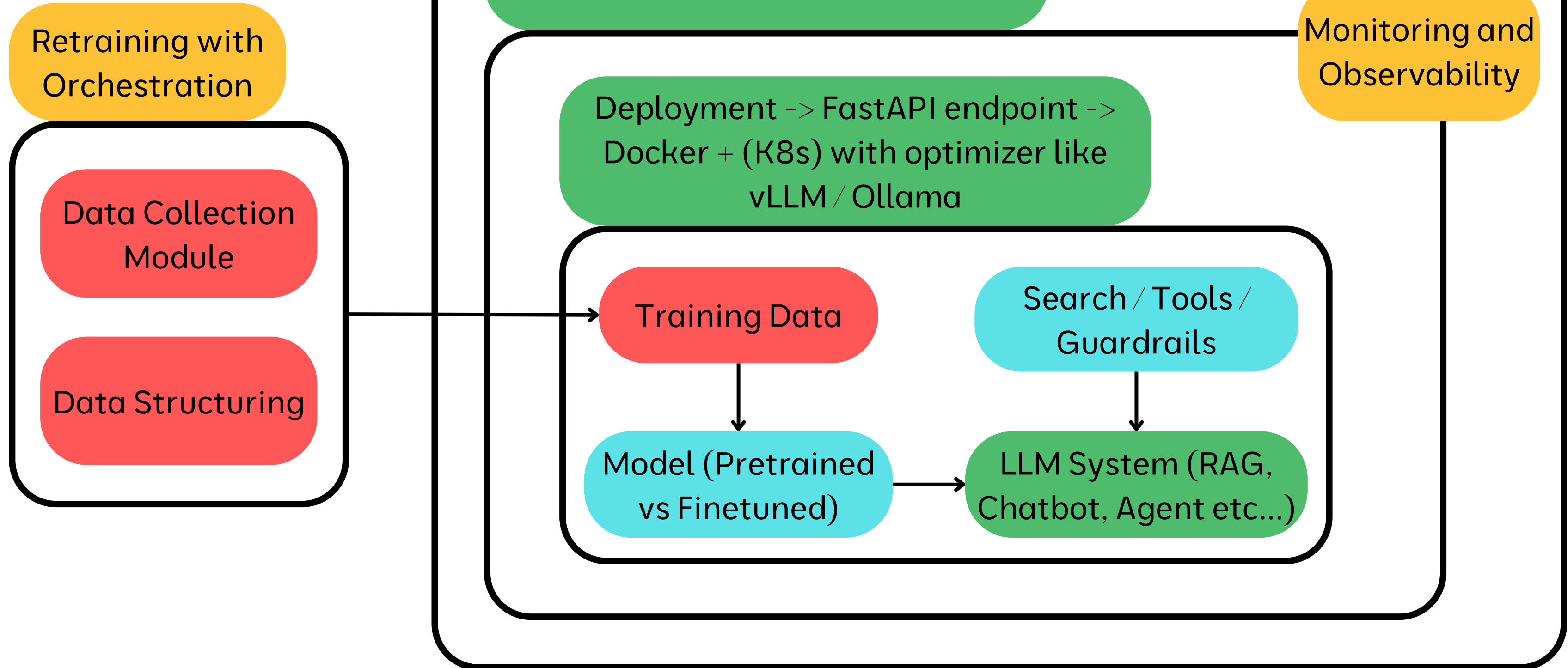
Setting Up Alerts & Anomaly Detection

Proactive issue resolution with automated alerts

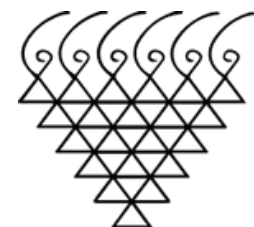
Development Flow



Example



Perplexity Clone



TMLC
ACADEMY